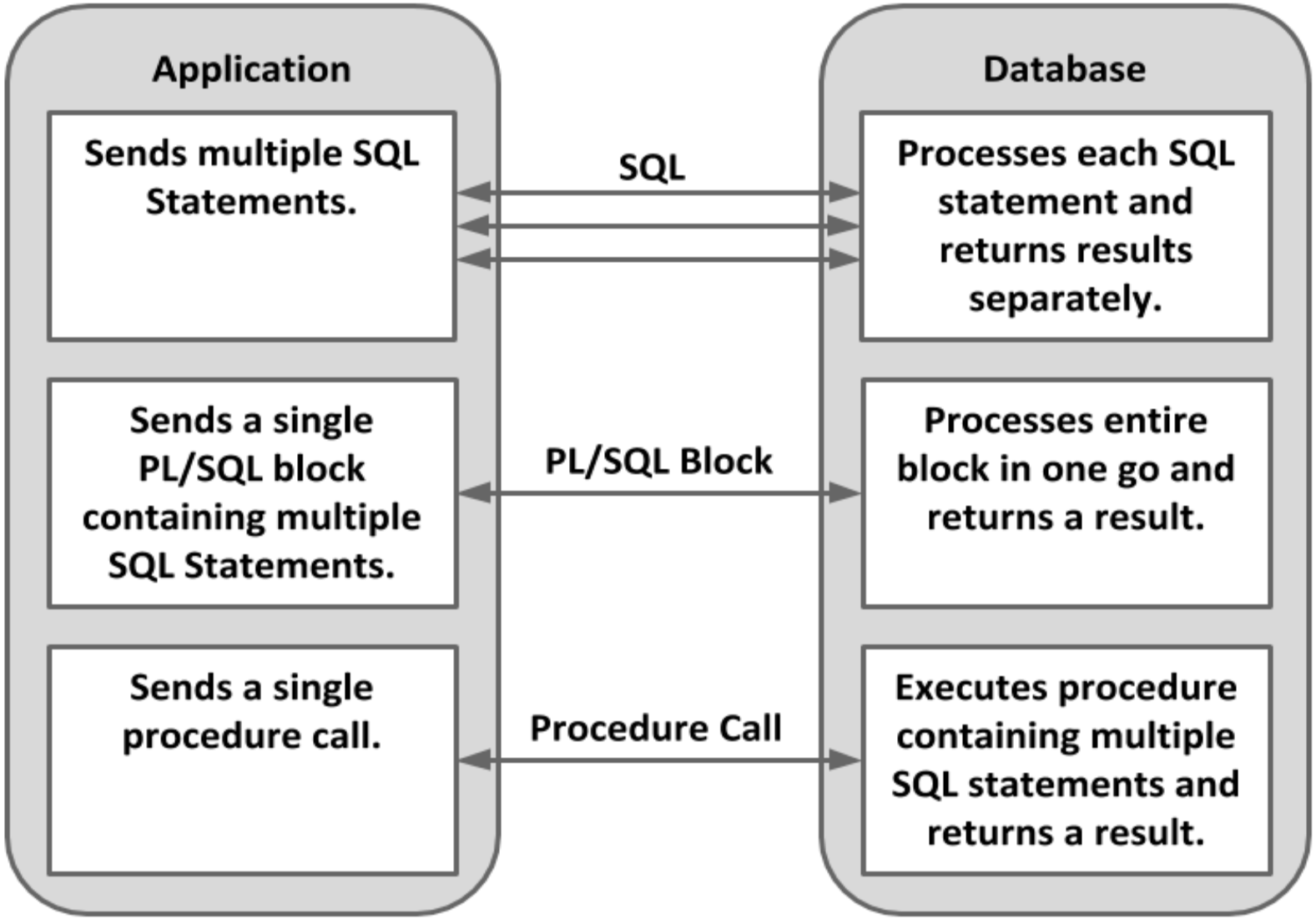
- PL/SQL Integration: Extends SQL procedurally, blending SQL seamlessly, unlike other languages needing complex APIs for data type mapping, statement preparation, and result processing.

- Data Types: PL/SQL uses database-compatible data types, minimizing conversions, unlike Java/.NET where handling database dates is complex.

- Efficiency: PL/SQL in the database reduces network overhead with single-call transactions, unlike middle-tier apps requiring multiple database interactions.


Application
Database
Sends multiple SQL Statements.
SQL
Processes each SQL statement and returns results separately.
Sends a single PL/SQL block containing multiple SQL Statements.
PL/SQL Block
Processes entire block in one go and returns a result.
Sends a single procedure call.
Procedure Call
Executes procedure containing multiple SQL statements and returns a result.

PL/SQL (Procedural Language/SQL) is Oracle's extension of SQL that adds procedural features like loops, conditions, and error handling.

It allows developers to write powerful programs that combine SQL queries with logic to control how data is processed.

With PL/SQL, complex operations, calculations, and error handling can be performed directly within the Oracle database, making data manipulation more efficient and flexible.

PL/SQL allows developers to:

Execute SQL queries and DML commands inside procedural blocks.

Define variables and perform complex calculations.

Create reusable program units, such as procedures, functions, and triggers.

Handle exceptions, ensuring the program runs smoothly even when errors occur.

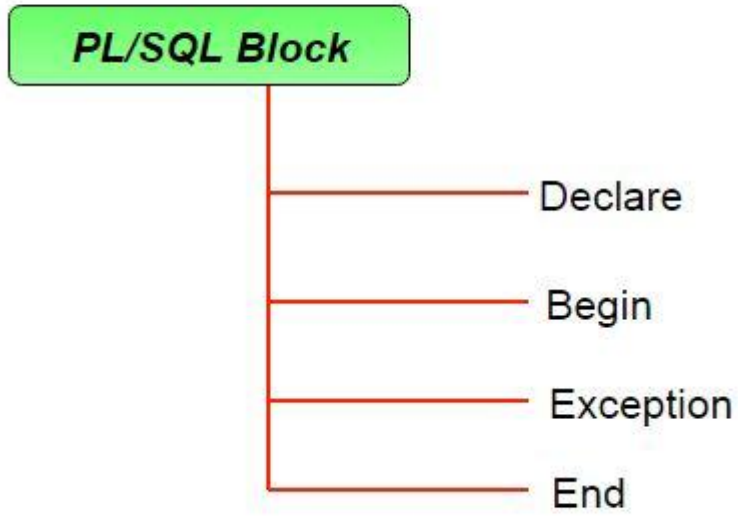
Key Features of PL/SQL

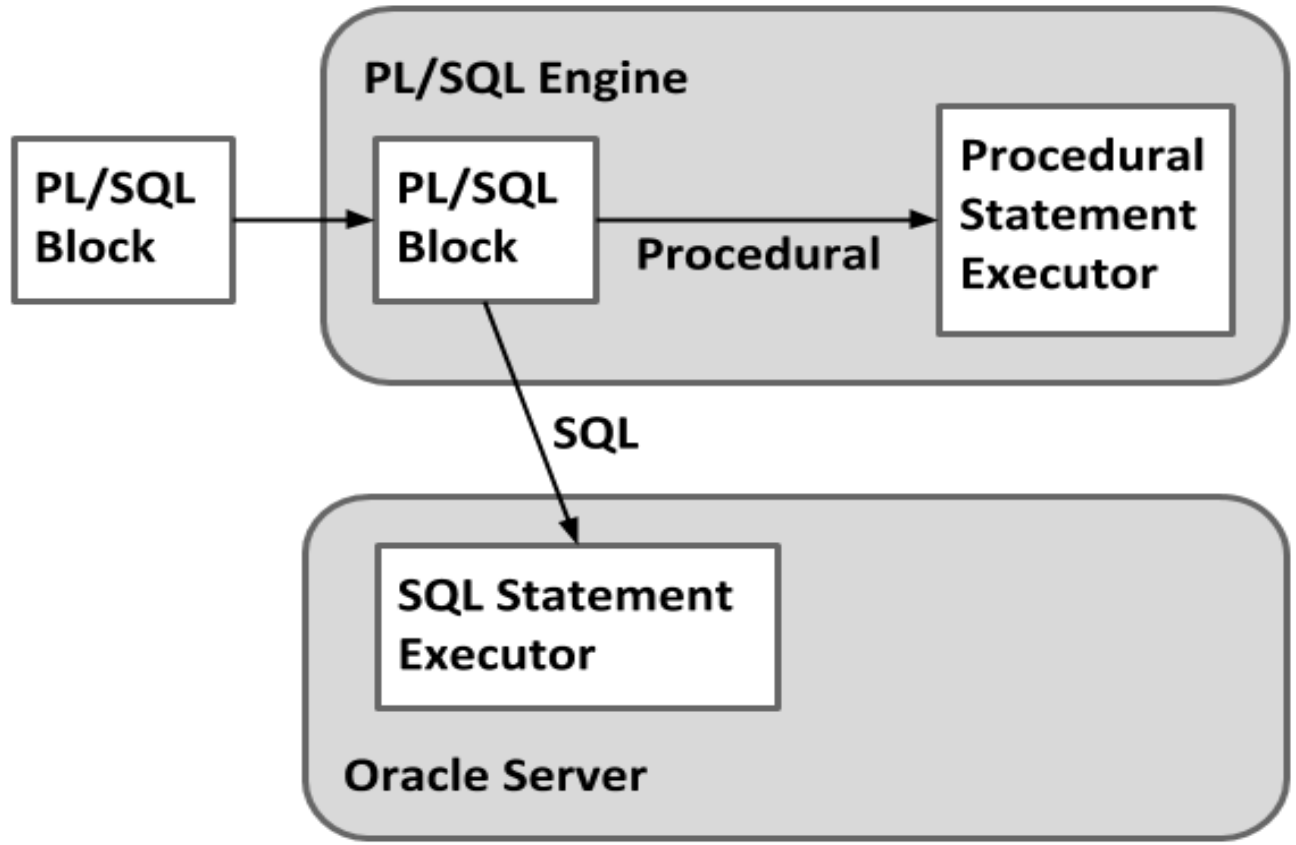
PL/SQL brings the benefits of procedural programming to the relational database world. Some of the most important features of PL/SQL include:

- **Block Structure:** PL/SQL can execute a number of queries in one block using a single command.
- **Procedural Constructs:** One can create a PL/SQL unit, such as procedures, functions, packages, triggers, and types, which are stored in the database for reuse by applications.
- **Error Handling:** PL/SQL provides a feature to handle the exception that occurs in a PL/SQL block, known as an exception handling block.
- **Reusable Code**: Create stored procedures, functions, triggers, and packages, which can be executed repeatedly.
- **Performance**: Reduces network traffic by executing multiple SQL statements within a single block

Structure of PL/SQL Block

PL/SQL extends SQL by adding constructs found in **procedural languages**, resulting in a structural language that is more powerful than SQL. The basic unit in PL/SQL is a block. All PL/SQL programs are made up of blocks, which can be nested within each other.





PL/SQL Architecture

PL/SQL code is written in blocks, which consist of three main sections:

•Declare section starts with **the DECLARE keyword,** in which variables, constants, and records as cursors can be declared, which store data temporarily. It basically consists definition of PL/SQL identifiers. This part of the code is optional.

•Execution section starts with **BEGIN** and ends with the **END** keyword. This is a mandatory section, and here the program logic is written to perform any task, such as loops and conditional statements. It supports all DML commands, DDL commands, and SQL*PLUS built-in functions as well.

•Exception section starts with the **EXCEPTION** keyword. This section is optional and contains statements that are executed when a run-time error occurs. Any exceptions can be handled in this section.

PL/SQL Identifiers

In PL/SQL, identifiers are names used to represent various program elements like variables, constants, procedures, cursors, triggers, etc. These identifiers allow you to store, manipulate, and access data throughout your PL/SQL code.

1. **Variables in PL/SQL**

Like several other programming languages, variables in PL/SQL must be declared prior to its use. A variable is like a container that holds data during program execution. Each variable must have a valid name and a specific **data type**.

Syntax for declaration of variables:

variable_name datatype [NOT NULL := value];

variable_name: The name of the variable.

datatype: The data type of the variable (e.g., INTEGER, VARCHAR2).

NOT NULL: This optional constraint means the variable cannot be left empty.

:= value: This optional assignment assigns an initial value to the variable.

2. Displaying Output in PL/SQL

The outputs are displayed by using DBMS_OUTPUT, which is a built-in package that enables the user to display output, debugging information, and send messages from PL/SQL blocks, subprograms, packages, and triggers.

3. Comments in PL/SQL

- Like in many other programming languages, in PL/SQL also, comments can be put within the code, which has no effect on the code. There are two syntaxes to create comments in PL/SQL :
- **Single Line Comment:** To create a single line comment, the symbol - - is used.
- **Multi-Line Comment:** To create comments that span over several lines, the symbols/* and */are used.

Feature	SQL	PL/SQL
Purpose	Single query used for DML (Data Manipulation Language) and DDL (Data Definition Language) operations.	Block of code used to write entire program blocks, procedures, functions, etc.
Nature	Declarative, defines *what* needs to be done.	Procedural, defines *how* things need to be done.
Execution	Executes a single statement.	Executes a block of code.
Use Case	Data retrieval, manipulation, and definition (e.g., SELECT, INSERT, UPDATE).	Used to create applications.
Syntax	Consists of SQL statements only.	Combines SQL statements with procedural logic.
Data Handling	Performs actions directly on the database.	Can contain SQL within its blocks, offering more control over data handling.